

Gebze Technical University

CSE 341 — Concepts of Programming Languages - Spring 2026

Project Part 2 — Example Programs & Test Report (D3) ScoreScript

Student Number: **230104004090**

Name: **FATİH EMRE OĞAN**

Submission Date: **22 May 2026**

This report documents the example programs and test cases for ScoreScript Part 2. Part A presents the three valid programs from Part 1, now executed end-to-end with their actual interpreter output. Part B presents the type-error program added in Part 2. Part C retains the five parse-error programs from Part 1.

Part A — Valid Programs (with Interpreter Output)

All three programs were tested using: `java -cp out Main <file>`

Program 1 — program1.ss

Focus: Student declarations, group declarations, variable declarations, all four domain operations

Constructs demonstrated: student, group, int variable, curve, average, rank, transcript

Interpreter Output

```
Average of ClassZ23: 77.17
Ranking of ClassZ23:
  1. Ayse KURT (96.33)
  2. Fatih Emre OGAN (87.33)
  3. Mehmet DEMIR (70.00)
  4. Zeynep ARSLAN (55.00)
Transcript for 230104004090 — Fatih Emre OGAN
  math: 90
  science: 95
  history: 77
  Average: 87.33
Transcript for 230104004091 — Ayse KURT
  math: 96
  science: 93
  history: 100
  Average: 96.33
Transcript for 230104004092 — Mehmet DEMIR
  math: 65
  science: 75
  history: 70
```

```
Average: 70.00
Transcript for 230104004093 — Zeynep ARSLAN
  math: 50
  science: 60
  history: 55
  Average: 55.00
Average of TopStudents: 91.83
Ranking of TopStudents:
  1. Ayse KURT (96.33)
  2. Fatih Emre OGAN (87.33)
```

Program 1 demonstrates the four core domain operations of ScoreScript working together. The curve operation adds 5 to every grade of every student in ClassZ23, clamping results to [0, 100] which is why Fatih Emre's math grade changed from 85 to 90 and history from 72 to 77 in the transcript output. The average operation then computes the mean of each student's grades and prints the group average of 77.17. The rank operation sorts students by their average in descending order, showing Ayse KURT at the top with 96.33. Finally, transcript prints each student's full record including their individual grades and personal average, confirming that curve mutated the stored grades before transcript ran.

Program 2 — program2.ss

Focus: User-defined functions with if/else if/else chains, bool variables, print statements

Constructs demonstrated: func, if/else if/else, bool variable, float variable, print, call statement, assignment

Note: float literals in letterGrade were updated from 90 to 90.0 etc. to satisfy the strict no-coercion type rule.

Interpreter Output

```
Transcript for 230104004090 — Fatih Emre OGAN
  math: 85
  science: 90
  english: 78
  Average: 84.33
Transcript for 230104004094 — Selin YILMAZ
  math: 42
  science: 55
  english: 60
  Average: 52.33
Transcript for 230104004095 — Burak SAHIN
  math: 95
  science: 98
  english: 92
  Average: 95.00
Average of SectionA: 77.22
Average of SectionA: 80.11
Ranking of SectionA:
  1. Burak SAHIN (97.67)
  2. Fatih Emre OGAN (87.33)
```

```
3. Selin YILMAZ (55.33)
true
false
BA
CC
Status: HONOR STUDENT
Status: PASSING
Status: FAILING
```

Program 2 demonstrates user-defined functions, conditional logic and bool variables working together. The transcript output shows the three students raw grades before any curve is applied .Fatih Emre averages 84.33, Selin 52.33 and Burak 95.00. The average is printed twice: 77.22 before the conditional curve and 80.11 after confirming that the if curveApplied == false branch executed and added 3 to every grade. The isPassing function returns true for 72.0 and false for 55.0 which is correct since passingGrade is 59.5. The letterGrade function correctly maps 88.5 to BA and 61.0 to CC using the if, else if chain and printLabel correctly identifies 92.0 as HONOR STUDENT, 72.0 as PASSING, and 45.0 as FAILING.

Program 3 — program3.ss

Focus: For loops over groups, assignment statements, curve with computed expression, nested if inside for

Constructs demonstrated: for student ... in, assign_stmt, curve by <expr>, multiple groups, if inside for body

Interpreter Output

```
Transcript for 230104004090 — Fatih Emre OGAN
  math: 85
  science: 90
  history: 72
  Average: 82.33
Transcript for 230104004091 — Ayse KURT
  math: 91
  science: 88
  history: 95
  Average: 91.33
Transcript for 230104004092 — Mehmet DEMIR
  math: 60
  science: 70
  history: 65
  Average: 65.00
Transcript for 230104004096 — Deniz KAPLAN
  math: 30
  science: 40
  history: 35
  Average: 35.00
Transcript for 230104004097 — Elif CELIK
  math: 78
  science: 82
  history: 80
  Average: 80.00
Average of MorningClass: 79.56
```

```
Ranking of MorningClass:
  1. Ayse KURT (91.33)
  2. Fatih Emre OGAN (82.33)
  3. Mehmet DEMIR (65.00)
Average of AfternoonClass: 57.50
Ranking of AfternoonClass:
  1. Elif CELIK (80.00)
  2. Deniz KAPLAN (35.00)
Ranking of AllStudents:
  1. Ayse KURT (97.67)
  2. Fatih Emre OGAN (89.33)
  3. Elif CELIK (84.00)
  4. Mehmet DEMIR (72.00)
  5. Deniz KAPLAN (39.00)
Transcript for 230104004090 - Fatih Emre OGAN
  math: 92
  science: 97
  history: 79
  Average: 89.33
false
Transcript for 230104004091 - Ayse KURT
  math: 98
  science: 95
  history: 100
  Average: 97.67
false
Transcript for 230104004092 - Mehmet DEMIR
  math: 67
  science: 77
  history: 72
  Average: 72.00
false
Average of AfternoonClass: 68.50
Ranking of AfternoonClass:
  1. Elif CELIK (91.00)
  2. Deniz KAPLAN (46.00)
Transcript for 230104004090 - Fatih Emre OGAN
  math: 92
  science: 97
  history: 79
  Average: 89.33
Transcript for 230104004091 - Ayse KURT
  math: 98
  science: 95
  history: 100
  Average: 97.67
Transcript for 230104004092 - Mehmet DEMIR
  math: 67
  science: 77
  history: 72
  Average: 72.00
Transcript for 230104004096 - Deniz KAPLAN
  math: 41
  science: 51
  history: 46
  Average: 46.00
Transcript for 230104004097 - Elif CELIK
```

```
math: 89
science: 93
history: 91
Average: 91.00
Average of AllStudents: 79.20
Ranking of AllStudents:
1. Ayse KURT (97.67)
2. Elif CELIK (91.00)
3. Fatih Emre OGAN (89.33)
4. Mehmet DEMIR (72.00)
5. Deniz KAPLAN (46.00)
```

Program 3 demonstrates for loops, multiple groups with different curve amounts and how assignment statements affect subsequent operations. The first section shows raw transcripts for all five students via a for loop over AllStudents then computes separate averages and rankings for MorningClass and AfternoonClass. MorningClass receives a curve of baseCurve + bonusCurve (4 + 3 = 7) while AfternoonClass receives only baseCurve (4) which is why Fatih Emre's math grade goes from 85 to 92 and Deniz KAPLAN's from 30 to 41 different curve amounts applied to different groups. The needsCurve function is called inside the for loop for MorningClass students and returns false for all three because their averages after curve are above 65.0. Finally baseCurve is reassigned to 5 and the if condition calls needsCurve(58.0) which returns true, triggering a second curve on AfternoonClass with the updated baseCurve + bonusCurve value.

Part B — Type Error Program

error_type.ss

Error type: void function result used as a value

The type checker catches this error before the interpreter runs. The program parses successfully (--dump-ast exits 0) but fails at type-check time (exit code 3).

Type Checker Output

```
Type error at line 45: cannot use the result of void function 'classReport' as a
value
```

Explanation:

The function classReport is declared with return type void. On line 45, the program attempts to assign its return value to a string variable: string summary = classReport(curveAmount). Since void functions produce no value, this is a type error. The type checker catches it statically before any execution, which is the correct behaviour under ScoreScript's strong typing rules (Sebesta §6.12).

Part C — Invalid Programs (Parse Errors, unchanged from Part 1)

File	Error Type	Line	Error Message
error1.ss	Wrong field order in student	10	expected 'firstName' but found 'lastName'
error2.ss	':' instead of '=' in group	22	expected '=' but found ':'
error3.ss	Missing '->' in func	17	expected '->' but found 'string'
error4.ss	Missing expr after 'by'	19	expected an expression but found 'average'
error5.ss	Statement before declarations end	18	unexpected token 'student' — expected a statement

All five parse errors are unchanged from Part 1. They are retained here to show that the Part 2 pipeline (TypeChecker + Interpreter added) does not affect parse-time error detection.